

From characters to next-token prediction

A fixed window of characters, learned embeddings, one hidden layer, and a distribution over the next token.

THE QUESTION

Given the characters so far, how can a network score every possible next character?

THE INTUITION

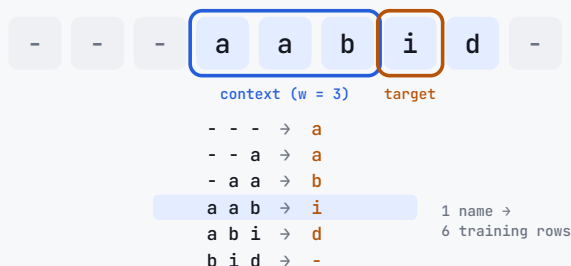
Turn the last few characters into learned numbers, mix them with an MLP, and read off one probability per token. Generating is just predicting, choosing, and sliding forward.

KEY TAKEAWAY

Next-token prediction is classification, repeated. A fixed window works, but it is baked into the first weight matrix.

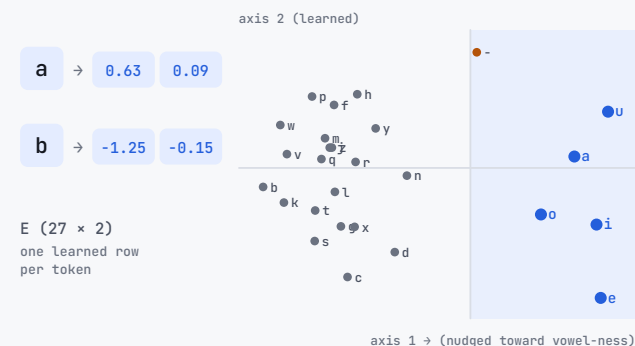
context ids → embedding lookup → concatenate → hidden ReLU → 27 logits → softmax → sample / loss

1 Turn names into training pairs



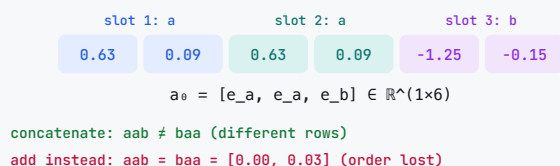
With a window of $w = 3$ tokens the model estimates $p_{\theta}(x_{t+1} | x_{t-2}, x_{t-1}, x_t)$. The input is the context; the target is the **observed next token**. The boundary - pads the start and ends a name, so the vocabulary is 26 letters + - = 27 tokens.

2 An ID selects a learned row



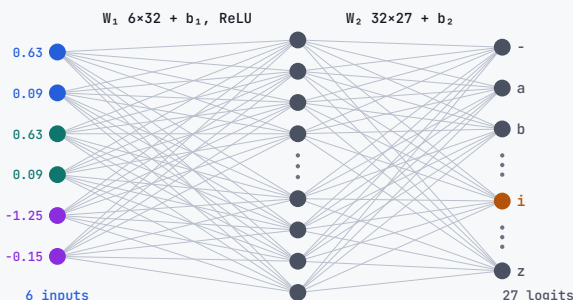
A token ID is an index, not a magnitude. The shared table $E_{\text{tok}} \in \mathbb{R}^{27 \times 2}$ returns two learned numbers per token. Both uses of a read the **same row**. Axis 1 was nudged toward vowel-ness during training so the picture is readable; learned axes need not have names.

3 Keep order by concatenating



One **six-number input**, not six tokens: each slot has its own weights into the hidden layer. Summing would discard which character sat where.

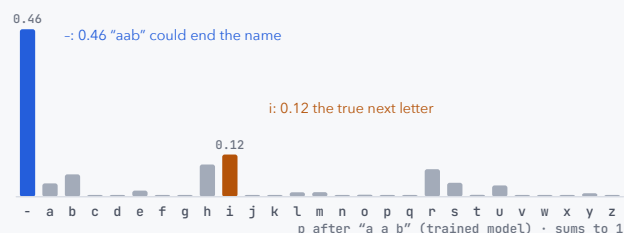
4 A hidden layer, then 27 scores



Row vectors throughout: $a_1 = \text{ReLU}(a_0 W_1 + b_1)$, $z = a_1 W_2 + b_2$. Hidden activations are recomputed for every context; they are not parameters. Without the nonlinearity, two affine layers collapse into one.

parameter	shape	count
E_{tok}	27×2	54
W_1	6×32	192
b_1	1×32	32
W_2	32×27	864
b_2	1×27	27
total		1,169

5 Scores become probabilities



$p_k = \exp(z_k - c) / \sum_j \exp(z_j - c)$ with $c = \max_j z_j$: nonnegative, sums to 1, and subtracting c avoids overflow without changing p . The model need not be certain; it should spread probability over plausible next characters.

What changes, and when?

Training changes parameters. Generation reuses them: predict, choose a token, slide the window.

6 Score the truth with cross-entropy



For target index y : $\ell = -\ln p_y = -z_y + \ln \sum_j \exp(z_j)$. Confident and right costs little; confident and wrong costs a lot. Over a batch, $\mathcal{L} = -\frac{1}{B} \sum_b \ln p_{b,y_b}$ (in nats); perplexity = $\exp(\mathcal{L})$, comparable only under the same tokenization.

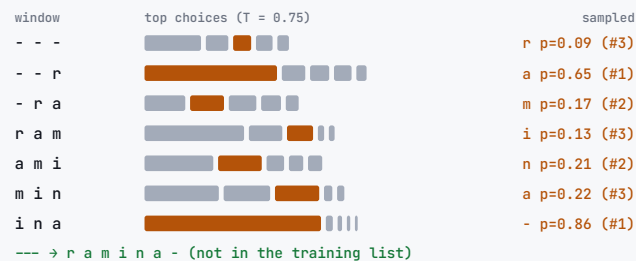
7 Backprop trains the whole model

Per example, $(\partial \ell) / (\partial z_k) = p_k - \mathbb{1}[k = y]$. The chain rule carries it through W_2 , ReLU, W_1 and the embedding lookups; SGD steps $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$ with $\theta = \{E_{\text{tok}}, W_1, b_1, W_2, b_2\}$. For $aab \rightarrow i$, both uses of a add gradient to the same row; the target i selects the loss but is not looked up.

```
optimizer.zero_grad()
z = model(context_ids) # [B, 27] logits
loss = F.cross_entropy(z, target_ids)
loss.backward()
optimizer.step()
```

Pass raw logits to cross-entropy. PyTorch accumulates gradients, so clear them. Split train/held-out by name before making windows.

8 Generate: sample, append, slide



Start from - - -, run the model, choose a token, append it, keep the last three, repeat until -. Greedy takes the argmax; sampling can pick a non-winner. Temperature: $p = \text{softmax}(z/\tau)$; small τ sharpens, large flattens. No parameter update happens.

9 Training is not generation

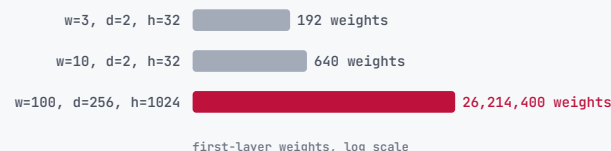
Training

context from data; the true next token is known; loss and gradients update parameters; many windows in a batch.

Generation

next token unknown; the chosen token becomes input; parameters stay fixed; one step at a time.

10 Why move on to attention?



Characters, words or subwords change the vocabulary, not the objective. But a fixed window cannot see clues outside it, and W_1 has wd rows: the window is baked into the network. Part 2: let each token read the relevant earlier tokens into a fixed-width update.

11 Pause and think: what job does each piece do?

Why not one-hot inputs? They identify tokens but give no compact learned geometry.

Why $-\log p$? It rewards probability on the observed target and punishes near-zero probability hard.

Why concatenate? It keeps position; a sum would forget which slot held which character.

What changes with w ? The first matrix: its input width is wd , so a new w is a new network.

Why 27 outputs? One logit per vocabulary token, so softmax can cover every possible next token.

What does generation need? A forward pass that training already made produce useful distributions.

IDs select rows · embeddings are learned · activations depend on the input · generation reuses a frozen model

Numbers from the trained name model bundled with the Part 1 interactive lesson (6,478 names; displayed values rounded). Watch the 3-minute summary video of this part.